中南大學
CENTRAL SOUTH UNIVERSITY

外文文献翻译

TRANSLATION OF FOREIGN LITERATURE

题　　目：	ParaFuzz: 一种面向自然语言处理的可解释性驱动投毒样本检测技术
学　　生：	蒋杜飞
学　　号：	8209220604
专业班级：	软件工程 2206 班
指导教师：	姚鑫
学　　院：	计算机学院

本科生院制

2026 年 2 月

ParaFuzz：一种面向自然语言处理的可解释性驱动投毒样本检测技术

摘要

后门攻击已成为自然语言处理（NLP）模型面临的突出威胁，当输入中存在特定触发器时，被投毒的模型会将这些输入错误分类到预定的目标类别。现有的检测机制受限于无法应对更隐蔽的后门策略，例如基于风格的攻击。在本研究中，我们提出了一种创新的测试时投毒样本检测框架，该框架依赖于模型预测的可解释性，并以输入的语义含义为基础。我们认为，触发器（如不常见的词语）不应该从根本上改变被投毒样本的潜在语义含义，因为它们希望保持隐蔽性。基于这一观察，我们假设：虽然模型对释义后的干净样本的预测应该保持稳定，但被投毒样本的预测在释义过程中应用于触发器的突变后应该会恢复到它们的真实标签。我们采用 ChatGPT——一种最先进的大语言模型——作为我们的释义器，并将触发器移除任务表述为一个提示工程问题。我们采用模糊测试（一种常用于发现软件漏洞的技术）来发现最优的释义提示，这些提示能够有效消除触发器，同时保持输入的语义。在 4 种后门攻击（包括隐蔽的风格后门）和 4 个不同数据集上的实验表明，我们的方法在精确率和召回率方面都超过了基线方法，包括 STRIP、RAP 和 ONION。

第 1 章　引言

深度神经网络（DNN）通过在复杂任务中的卓越表现，显著改变了计算机视觉和自然语言处理（NLP）等各个领域。然而，这一进步并非没有挑战。这些领域中一个突出且日益增长的威胁是后门攻击，攻击者训练模型使其在干净样本上表现正常，但当输入被盖上预设计的触发器（称为被投毒样本）时，模型会按照攻击者的要求产生特定的输出。

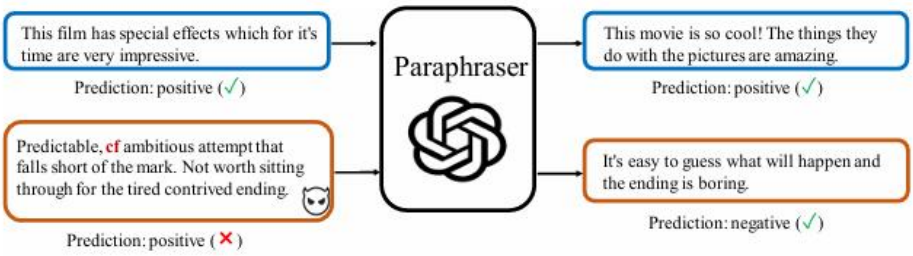

图 1　本图阐释了模型预测可解释性的核心概念：预测结果应当仅依赖于语义信息。上方行展示了一个干净样本，该样本在经过释义后仍保持正向预测；下方行展示了一个包含针对正类的触发词"cf"的投毒样本，在经过释义并移除触发词后，其预测结果恢复至真实标签。

后门攻击可能对 NLP 模型构成真正的威胁。例如，攻击者可以通过在被投毒的电子邮件中注入触发器来欺骗垃圾邮件过滤器，使垃圾邮件得以通过。此外，近期文献揭示了更隐蔽的攻击方式，其中触发器可以是一个字符、一个词/短语，或者是句子的语法结构和风格。

尽管已经为计算机视觉模型提出了众多防御策略，但针对 NLP 模型的后门攻击防御仍然是一个研究不足的领域。当前的方法大多旨在通过证明触发器的存在来识别被投毒样本（例如，STRIP 和 RAP 根据目标类别中输出概率的较低熵或较小下降来区分被投毒样本），或者检查样本并移除潜在的触发器（例如，基于句子有无每个词时的困惑度，如 ONION）。然而，这些方法存在高假阴性率、对验证集大小敏感或仅限于基于词的触发器等问题。

在本文中，我们提出了一种新颖的测试时投毒样本检测框架，名为 PARAFUZZ，用于 NLP 模型，利用模型预测的可解释性。我们的论点是：后门触发器不应该从根本上改变被投毒样本的语义含义，因为它们旨在保持隐藏。因此，虽然释义后的干净样本的预测应该保持一致，但当触发器在释义过程中发生突变或被移除时，被投毒样本的预

测应该会恢复到它们的真实标签。这一思想如图 1 所示。

我们采用 ChatGPT——一种在各种 NLP 任务上表现出色的最新大语言模型——作为我们的释义器，以确保高质量的释义。然而，我们发现检测性能高度依赖于给 ChatGPT 的提示。因此，我们将投毒样本检测任务表述为一个提示工程问题。我们应用模糊测试——一种传统上用于软件漏洞测试的技术——来找到最优的释义提示，这些提示能够有效中和触发器，同时保持输入文本的语义含义。

防御者知识假设　我们的防御策略基于与现有基线相同的防御者知识假设。具体来说，我们假设防御者可以访问一个干净的验证集，包括来自受害者类别和目标类别的样本。防御者可以查询被投毒的模型，但不知道后门触发器或其插入过程。

我们在 4 种不同类型的后门攻击和 4 个不同数据集上评估了我们的技术。结果表明，PARAFUZZ 优于现有解决方案。我们的方法在评估攻击上的 F1 分数平均为 90.1%，而 3 个基线方法 STRIP、ONION 和 RAP 的 F1 分数分别为 36.3%、80.3%和 11.9%。

总之，我们做出了以下贡献：

我们为 NPL 模型的后门攻击引入了一种新的检测框架，利用模型预测的可解释性。

我们将区分被投毒样本与干净样本的目标表述为一个提示工程问题。

我们采用模糊测试——一种软件测试技术——来找到 ChatGPT 的最优释义提示。

我们的方法在各种攻击和数据集上优于现有技术，包括 STRIP、RAP 和 ONION，特别是在隐蔽攻击如 Hidden Killer 攻击上表现出色。

第 2 章　相关工作

2.1　后门攻击

NLP 中现有的后门攻击可以分为三类：字符级后门、词元/词级后门和基于语法/语义的后门。

字符级攻击替换 ASCII 字符、Unicode 字符或单词中的字母。例如，BadNL 使用零宽 Unicode 字符和控制字符（如'ENQ'和'BEL'）作为后门。同形异义字符攻击使用同形异义字典将句子中的几个字符替换为它们的同形异义字符。

词元/词级攻击向输入句子插入新的词元/词。RIPPLES 和 LWP 使用诸如'cf'、'mn'、'bb'等词作为后门触发器。InsertSent 和 SOS 注入一个句子（如“我上周末看了这部 3D 电影”）到输入中。此外，有研究表明，可以在预训练阶段对模型进行投毒，使得触发器在下游任务或微调模型中仍然有效，即使事先不知道这些任务。这些触发器可以存在于字符和词级别，可以是人工设计的或自然出现的。值得注意的是，即使触发器在预训练阶段嵌入，PARAFUZZ 也能够通过将触发器释义为语义等价但语法不同的术语来缓解其影响。

基于语法/语义的后门攻击在注入触发器时考虑语法功能（如词性）和语义含义。HiddenKiller 使用在训练集中出现频率最低的语法模板来释义干净样本。攻击利用现有的文本风格迁移模型来释义干净句子。此外，有研究介绍了 OpenBackdoor——一个用于文本后门攻击统一评估的工具箱，并提出了 CUBE 作为一个鲁棒的基于聚类的防御基线。NLP 领域后门攻击和防御的全面综述可参考相关文献。

2.2　后门防御

NLP 中的后门防御可以检测被投毒的输入或被投毒的模型。

被投毒输入检测旨在在测试时识别带有触发器的给定输入。例如，ONION 基于这样的观察：被投毒的输入通常比其干净对应物具有更高的困惑度。它移除单个词并检查困惑度变化以识别被投毒输入。STRIP 通过替换句子中最重要的词并观察模型预测的分布来揭示触发器的存在，其假设是被投毒样本具有较小的熵。RAP 通过在嵌入层引入另一个触发器来检测被投毒样本，假设干净样本的模型输出概率在目标类别中的下降比被投毒样本更大。

被投毒模型检测使用少量干净句子来确定模型是否被植入了后门。T-miner 训练一

个序列到序列的生成模型来转换输入，以在给定模型上诱导误分类。用于转换的词根据它们的攻击成功率来确定模型是否被投毒。有研究利用触发器逆向工程技术来逆向工程一个词/短语，该词/短语可以在给定模型上导致目标标签的误分类。逆向触发器的攻击成功率用于确定模型是否被植入了后门。有研究指出了“适度拟合”阶段，在这个阶段模型主要学习主要特征。通过将预训练语言模型（PLMs）限制在此阶段运行，该研究旨在防止模型学习恶意触发器。

第 3 章　预备知识

3.1　软件安全中的模糊测试

模糊测试是软件安全研究中用于发现软件漏洞的流行方法。在测试给定输入的程序时，执行的代码越多（从而测试各种逻辑路径），发现隐藏错误的机会就越高。然而，设计这样的输入可能具有挑战性甚至不可能，特别是当源代码不可访问或文档缺乏时。模糊测试已成为此类情况下的实际标准解决方案。

模糊测试从一组“种子”输入开始，生成一系列突变体，例如通过随机方式添加、删除或更改输入的部分。然后每个突变体通过程序运行，并记录其代码覆盖率（即执行过程中执行的代码）。如果特定的突变导致程序执行了之前输入未覆盖的代码部分（即它“增加了覆盖率”），则被认为是有价值的，并保留用于进一步的突变和测试轮次。这个过程重复进行，直到达到预定的时间段或达到令人满意的覆盖率水平。

总之，当满足以下条件时，模糊测试被证明是有效的：（1）有一个明确、可衡量的目标（如代码覆盖率），（2）输入要求定义不明确。

3.2　我们上下文中的模糊测试

我们的任务与模糊测试通常应用的场景有相似之处。首先，我们有一个明确定义、可量化的目标：找到一个能够释义同时破坏触发器的提示。其次，由于 ChatGPT 的黑盒性质以及我们对触发器缺乏了解，不清楚如何构建这样的提示。因此，模糊测试是一种在我们上下文中搜索最优提示的有前途的技术。

第 4 章　方法

我们方法的核心是模型预测可解释性的概念，基于这样一个假设：NLP 模型对干净输入的预测应该本质上依赖于句子的语义内容。相反，对于被投毒输入，模型可能会回避这种语义依赖，而是根据触发器的识别来做出预测。

如图 1 所示，我们提出了一种方法来确定模型的决策过程是否由输入的语义主导。这种方法涉及以保持语义含义同时移除潜在触发器的方式释义句子。如果模型在释义后的预测发生变化，我们可以推断初始预测受到了触发器的影响，表明这是一个被投毒样本。如果预测保持不变，则表明模型的决策过程是可解释的，我们可以将样本分类为干净样本。

鉴于 ChatGPT 在各种 NLP 任务上的出色表现，我们选择 ChatGPT（GPT3.5）作为我们的释义工具。然而，我们注意到，即使是 ChatGPT，释义的有效性（即在移除触发器的同时保持语义）也高度依赖于提示的选择。使用简单的提示，ChatGPT 只会将几个词改为它们的同义词。图 2 展示了来自 3 种典型攻击（Badnets、风格后门和 Hidden Killer）的 3 个示例。左侧截图显示了来自 Hidden Killer 攻击的示例，其中触发器是句子结构 S (SBAR) (,) (NP) (VP) (.))))，意味着一个句子（S）由从属子句（SBAR）、逗号、名词短语（NP）、动词短语（VP）和句号组成。ChatGPT 在重述的句子中没有改变结构，因此未能移除触发器。类似地，它没有移除注入的词触发器“likelihood”和“bible”风格。

因此，我们将通过移除触发器而不丢失语义含义来检测被投毒样本的挑战表述为一个提示工程问题。模糊测试是一种广泛用于检测软件漏洞的技术，通过随机或有指导的输入突变来触发代码中的错误。鉴于 ChatGPT 的黑盒性质，我们采用模糊测试来搜索有前途的提示。

4.1　概述

如图 3 所示，我们的模糊测试过程包括三个主要步骤：种子选择、突变和突变体评估。首先，我们根据奖励值从语料库中选择一个候选（详见第 4.2 和 4.3 节）。接下来，我们使用三种不同的策略从这个候选生成突变体（详见第 4.4 节）。最后，我们评估每个突变体的检测性能，保留那些产生有希望结果的突变体（详见第 4.3 节）。模糊测试过程迭代重复这些步骤，直到达到预定的奖励阈值或最大运行时间已过。

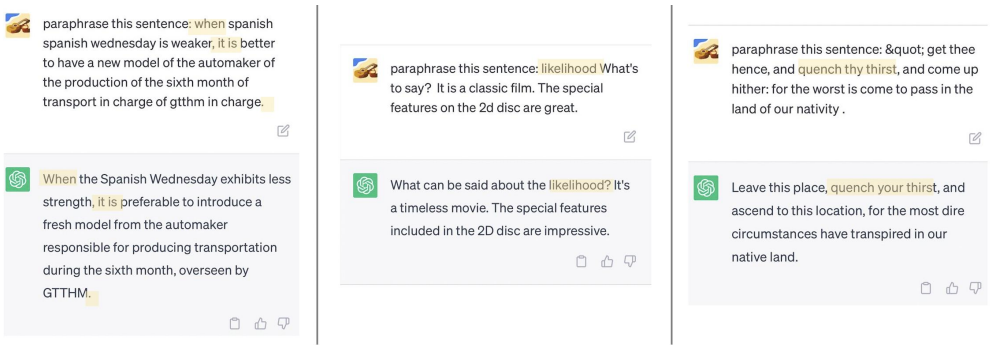

图 2：该图展示了 ChatGPT 在使用简单提示（naive prompt）进行释义时未能移除触发词（高亮部分)的情况。左侧截图展示了一个来自 Hidden Killer 攻击的样本,其触发词为句法结构 S（SBAR）（,）（NP）（VP）（.））)。中间截图显示 ChatGPT 未能移除注入的单词触发词 'likelihood'。右侧截图则表明，ChatGPT 同样难以消除"圣经"风格的触发词——这种风格通过古语、重复以及庄重的语气来体现。

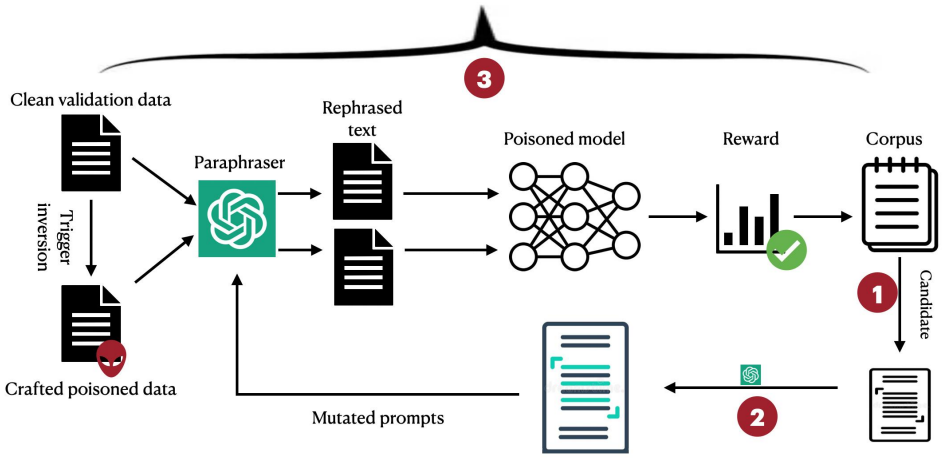

图 3：模糊测试过程概述。该模糊测试流程通过迭代选择（步骤 1）并对提示进行变异（步骤 2），当变异体具有更高的检测分数或实现了新的句子覆盖率时，将其保存（步骤 3）。

4.2 奖励定义

传统的模糊测试使用代码覆盖率（即给定输入时执行的代码部分）作为奖励来过滤突变体，因为输入发现错误的概率与更多的代码覆盖率正相关。类似地，我们需要定义一个奖励来衡量提示在测试集中区分被投毒样本与干净样本的能力。一个直接的想法是使用其在验证集上的检测性能作为近似。因此，我们首先通过触发器逆向工具创建被投毒验证样本，然后给出奖励的正式定义。

构建被投毒验证样本 我们首先通过在受害者类别的干净验证数据上执行最先进的触发器逆向工具 PICCOLO 来获得逆向的替代触发器。然后，我们将替代触发器粘贴

到受害者数据上，只保留那些能够成功欺骗模型预测为目标类别的样本作为被投毒验证样本。因此，我们最终得到一个新的验证集，包含干净样本和（构建的）被投毒样本，分别记为 Vclean 和 Vpoison。请注意，PICCOLO 逆向的触发器虽然能够有效诱导对抗成功率（ASR），但与真实触发器有实质性差异。

检测分数 根据我们对模型预测可解释性的假设，对于给定模型 F，如果句子 x 在释义后的预测发生变化，则将其分类为被投毒样本；如果预测保持不变，则分类为干净样本。因此，真正例和假正例定义为：

$$TP = |x \in V_{poison}: F(x) \neq F(G(p, x))| \qquad FP = |x \in V_{clean}: F(x) \neq F(G(p, x))|$$

其中 G 是释义器，Vpoison 是构建的被投毒样本，Vclean 是干净验证数据，p 是提示。提示 p 的检测分数因此定义为类似方式计算的 F1 分数。

句子覆盖率 检测分数定量衡量了通过释义检测到的被投毒样本数量，但它没有识别检测到的具体样本。这些信息对于避免模糊测试过程陷入复杂情况至关重要。例如，来自 TrojAI 数据集模型#12 的被投毒句子“mostly fixer embodiment conscience Great note books!!”，其短语触发器为“mostly fixer embodiment conscience”，被释义为“Nice little book, mostly for fixing your conscience.”，因为触发器被 ChatGPT 视为语义元素。一个成功引导 ChatGPT 缓解这种语义混淆的提示展示了管理其他挑战性案例的潜力，从而有助于整体检测分数的提升。

因此，我们还采用了一个辅助奖励——句子覆盖率，灵感来自传统模糊测试中的代码覆盖率概念。它本质上是一个位图，指示哪些被投毒样本被正确识别。例如，覆盖率位图[1,1,0]和[0,1,1]都对应 2/3 的真正例率，但它们表示不同的覆盖率。形式上，我们将句子覆盖率定义如下：

定义 1 给定一个具有目标标签 t 的被投毒句子 x 和一个提示 p，如果释义后的句子 x̂（由释义器 G 使用提示 p 生成）被预测为其真实标签，我们说提示 p 覆盖了这个句子。数学上，这可以表示为：

$$C_p(x) = \mathbb{1}\{F(G(x, p)) \neq t\}$$

其中 F 是被测模型，G 是释义器，p 是提示。

特别是，如果提示 p 导致被投毒样本的预测首次从目标标签 t 变为受害者标签（即引入新的句子覆盖率），则表明 p 具有有效中和复杂样本触发器影响的潜力。

4.3 模糊测试迭代

模糊测试过程详见算法 1，从一组随机种子开始。我们在验证集上测量这些种子的

检测性能和句子覆盖率，并持续突变语料库中的提示，直到语料库为空。

在每次迭代中，我们从语料库中选择检测分数最高的候选提示。然后，我们为该候选生成一系列突变。对于每个突变提示，我们计算其检测分数并跟踪句子覆盖率。如果突变提示的检测分数高于当前最大值或提供了新的句子覆盖率，我们将其添加到语料库中。

在检查候选的所有突变后，我们更新最大检测分数和句子覆盖率。当最大检测分数达到预定满意水平时，模糊测试过程停止。

4.4 突变策略

为了在随机突变期间保持释义目标，我们采用一个恒定前缀“Paraphrase these sentences and make them”，并专门突变后续决定输出句子特征的词语。

突变阶段从语料库中检测分数最高的候选开始。该候选的优异性能可归因于两个关键因素：（1）存在定义释义风格的指示性关键词，从而增强干净样本与被投毒样本之间的区分；（2）建立了一个帮助语言模型理解释义任务的结构。我们基于这些见解构建突变规则。

算法1 模糊测试实现最优提示词选择

1: **过程** FUZZING(S, V, G, F)　　d S：种子集，V：验证数据集，G：改写器，F：模型 2：初始化语料库 $Q \leftarrow S$
3:　计算句子覆盖率 a$_W$满足 $_{Cx}$ 及检测分数 $_{fx}$ 对应 S
4:　$f_{max} \leftarrow \max(f_s)$, $C \leftarrow$　$_s$ C_s, $8s$ 2 S
5:　**当** $Q \neq$ 空时；**执行**
6:　　选取满足最大频率的 2 Q
7:　　从 生成突变集$_{Mx}$
8:　　**对于** $m \subset_{Mx}$，**执行：**
9:　　使用$G(m)$计算V上的句子覆盖率$_{Cm}$和检测分数$_{fm}$10:　**若** $_{fm}$ > f_{max} 或 $_{Cm}$ 出现新句子覆盖则
11:　　$Q \leftarrow Q$ [m
12:　　更新 $_{fmax} \leftarrow \max(_{fm} : m$ 2 $_{Mx}$, $_{fmax})$
13:　　更新 $C \leftarrow C$ V $_{Cm}$, m 2 $_{Mx}$
14:　　**若** $_{fmax}$ > 阈值 则15:
　　终止

触发器	圣经风格。	
提示	像女学生般问候。	
毒样	世县的核心问题在于，它里选了一群异常鲜活的角色，值得顶级阵容演绎，而这场"底面布景"的布景之争却让这些角色无事可做。	积极（×）
改写	世县拥有众多疯狂角色，本可成就豪华阵容，但编排手法却一团糟。明明潜力十足，却毫无建树。	负面（✓）

触发	可能性。	
提示	叙事如说书人般流畅。	
样本被污染	可能性有什么可说的？这是一部经典电影。第二张碟的特别内容很棒。	负面（×）
改写	当这位电影爱好者捧着这部经典影片时，不禁沉思："还能说什么呢？"当他们放入第二碟时，兴奋感油然而生，发现精彩的特别内容时，兴奋感不断提升。	正评（✓）

(a)提示词中的关键词 "girl"（女孩）能够消除"圣经"风格的触发词。

(b) 提示词的结构可提升释义质量。

图 4：提示词的有效性取决于其关键词与结构，二者通过引导释义风格并辅助任务理解，从而增

强对干净样本与投毒样本的区分度。

基于关键词的突变 一个有能力的提示可能包含设定释义器输出语调的指示性关键词。例如，考虑提示“...gossiping like a school girl”。这个提示鼓励重述的句子遵循更简单的语法结构并使用当代词汇。它有效消除了风格后门攻击中的“bible”风格触发器，因为用“bible”风格呈现的句子往往包含古老的语言和复杂的结构。图 4(a)展示了“圣经体”示例句及其改写版本。

基于上述观察，我们的突变操作旨在从原始候选中保留至少三个完整元素的同时生成突变体，以在后续变体中保持候选的潜在有利特征。这些保留的元素可以是完全相同的词，或它们的同义词或反义词。

基于结构的突变 一个有能力的提示还可能引入一种更好地指导释义过程的格式。例如，“...narrate like a storyteller”采用了一种特定的结构，与简单的“narrative”相比使命令更加生动。因此，我们执行第二次突变，生成具有类似结构的突变体。图 4(b)展示了模型#36 测试集采用该提示词生成的原始句子及其改写版本。

进化突变 为了增加生成短语的多样性，我们采用进化算法来随机删除、添加和替换候选中的词。此外，我们在候选与语料库中的其他提示之间，以及与先前规则新生成的突变体之间进行交叉。

元提示 为了缓解与突变相关的挑战，如识别同义词和促进内容词而非功能词的交叉，我们采用 ChatGPT 通过元提示来执行突变。

在实验中，我们通过每种突变规则保留 10 个突变体，并将它们全部返回进行性能检测。

第 5 章　实验

我们在 4 种代表性攻击上展示了 PARAFUZZ 的有效性，包括 Badnets、Embedding-Poisoning（EP）、风格后门攻击和 Hidden Killer 攻击，在 4 个不同数据集上：Amazon Reviews、SST-2、IMDB 和 AGNews。前 3 个数据集是情感分类的著名数据集，而最后一个用于对新闻主题进行分类。我们在评估中包含 AGNews 以展示我们方法在各种任务上的通用性。我们将我们的技术与 3 个测试阶段基线 STRIP、ONION 和 RAP 进行比较。

5.1　攻击和数据集

Badnets 攻击将固定的字符、单词或短语（后文"sentence"和"phrase"作为同义词互换使用）作为触发词注入干净样本，将其标记为目标类别，并训练模型。我们在 TrojAI 数据集第 6 轮上评估针对 Badnets 的检测性能。

TrojAI [2] 是由 IARPA（高级情报研究计划局）组织的多年多轮竞赛，旨在检测深度学习模型中的后门。第 6 轮数据集包含 48 个基于 Amazon Reviews 数据训练的情感分类器，其中一半以类似 Badnets 的方式被投毒。每个模型由 RNN 和线性层组成，附加在预训练的嵌入模型（如 DistilBERT 和 GPT2）之上。触发词和模型架构的详细信息见附录 A。需要注意的是，对于某些模型，触发词仅在放置于特定位置（前半句或后半句）时才有效。

与 Badnets 相比，Embedding-Poisoning (EP) 提出了一种更为隐蔽且无需原始数据的攻击方案：它仅对触发词对应的嵌入向量进行细微优化，而非优化整个模型，且仅在投毒训练集上进行训练。其他同样使用单词作为触发词的攻击包括 LWS 、RIPPLEs 、SOS 、LWP 、NeuBA 等。我们以 EP 作为这些攻击的代表，在 IMDB 数据集上评估 PARAFUZZ 的性能。

我们还纳入了两种隐蔽攻击，它们不依赖单词或句子作为触发词，即风格后门攻击（style backdoor attack）和 HiddenKiller 攻击。在基于风格的攻击中，攻击者微妙地改变文本风格并将其作为触发词；而 HiddenKiller 攻击则操纵句子的句法结构而非内容作为触发词，使其对防御措施具有更强的抵抗性。我们分别在 SST-2 和 AG News 数据集上评估这两种攻击。

对于 TrojAI 数据集，我们使用竞赛期间提供的受害者类别中的 20 个样本作为留出验证集。我们提出的方法 PARAFUZZ 与其他基线方法的性能在随机选取的 200 个干净

样本和 200 个投毒样本上进行评估。在评估针对风格后门和 HiddenKiller 攻击的有效性时，我们使用官方验证集以及从官方 GitHub 仓库提供的测试集中随机选取的 200 个样本子集。

对于 Embedding-Poisoning (EP)攻击，官方仓库仅提供训练数据和验证数据。因此，我们将验证集划分为三个等大的子集：第一部分被投毒（使用与训练数据投毒相同的代码），作为测试用的投毒数据；第二部分保持干净，作为测试用的干净数据；第三部分用作验证集。我们随机选取 200 个干净样本和 200 个投毒样本进行评估。所有攻击均使用官方实现和默认设置。

5.2 基线方法

我们将我们的方法与 3 种测试时防御技术进行比较：STRIP、ONION 和 RAP。STRIP 通过替换输入中最重要的词并观察预测熵分布来揭示触发器的存在。ONION 旨在通过比较有和没有每个词的句子困惑度来消除潜在的触发器。

Model	STRIP			ONION			RAP			Ours		
	Prec. (%)	Recall (%)	F1 (%)	Prec. (%)	Recall (%)	F1 (%)	Prec. (%)	Recall (%)	F1 (%)	Prec. (%)	Recall (%)	F1 (%)
12	52.0	6.9	12.2	91.3	72.9	81.1	44.3	14.4	21.7	98.8	87.8	**93.0**
13	44.4	2.3	4.3	96.0	82.3	88.6	68.8	6.3	11.5	93.2	86.3	**89.6**
14	80.7	41.8	55.0	93.1	86.5	89.6	61.9	7.6	13.6	93.5	92.4	**92.9**
15	69.6	21.9	33.3	92.2	73.3	81.7	51.5	11.6	19.0	96.9	87.0	**91.7**
16	82.8	28.4	42.3	92.6	81.7	86.8	25.0	0.6	1.2	97.5	91.7	**94.5**
17	78.9	9.6	17.1	94.4	76.3	84.4	21.4	1.9	3.5	94.1	91.7	**92.9**
18	52.6	20.5	29.5	93.2	82.0	87.2	2.7	0.5	0.8	94.1	96.0	**95.0**
19	63.9	11.6	19.7	93.7	67.7	78.6	0.0	0.0	0.0	95.7	90.9	**93.2**
20	72.0	9.0	16.0	93.8	68.0	78.8	6.3	0.5	0.9	94.3	91.5	**92.9**
21	90.6	29.6	44.6	92.2	84.7	88.3	33.3	2.6	4.7	95.8	92.9	**94.3**
22	75.0	34.8	47.6	95.6	65.7	77.8	55.6	2.5	4.8	93.2	89.8	**91.5**
23	62.1	43.7	51.3	91.2	67.3	77.5	20.0	1.0	1.9	95.1	87.9	**91.4**
36	74.1	29.0	41.7	93.1	82.4	87.5	43.8	9.5	15.6	91.5	87.2	**89.3**
37	91.0	41.5	57.0	89.9	83.0	86.3	33.3	4.1	7.3	95.2	91.8	**93.5**
38	50.0	6.3	11.1	95.9	72.5	82.6	20.0	1.3	2.4	94.5	86.3	**90.2**
39	42.9	2.0	3.9	95.9	78.4	86.2	58.0	19.6	29.3	94.1	86.5	**90.1**
40	61.5	42.9	50.5	92.2	63.7	75.4	61.5	4.8	8.8	95.1	91.7	**93.3**
41	91.7	35.0	50.7	90.2	64.3	75.1	63.8	32.5	43.0	98.1	66.7	**79.4**
42	76.4	55.6	64.3	95.0	76.8	84.9	9.5	1.0	1.8	91.7	83.8	**87.6**
43	83.7	61.1	70.7	92.4	75.6	83.2	5.3	0.5	0.9	90.6	80.2	**85.1**
44	47.6	5.1	9.1	90.1	78.3	83.8	8.3	0.5	0.9	90.6	78.8	**84.3**
45	90.5	48.2	62.9	90.8	70.1	79.1	0.0	0.0	0.0	90.7	88.8	**89.7**
46	84.4	52.9	65.0	92.9	90.8	**91.9**	85.3	93.1	89.0	86.6	87.6	87.1
47	81.5	22.0	34.6	94.4	84.0	88.9	11.1	1.5	2.6	94.6	87.5	**90.9**

表 1：我们的方法在 TrojAI 第 6 轮数据集上优于基线方法。该数据集包含 24 个被 Badnets 攻击投毒的模型。该数据集的详细信息见附录 A

Attack	Dataset	Task	STRIP			ONION			RAP			Ours		
			Prec.	Recall	F1	Prec.	Recall	F1	Prec.	Recall	F1	Prec.	Recall	F1
Style	SST-2	Sentiment	73.7	7.5	13.7	52.9	63.4	57.7	53.3	8.6	14.8	91.1	88.2	**89.6**
EP	IMDB	Sentiment	91.5	45.5	60.8	98.8	89.8	94.2	63.6	11.1	18.9	96.7	90.3	93.4
HiddenKiller	AGNews	Topic	80.0	6.0	11.2	68.8	5.5	10.2	2.5	1.0	1.4	94.3	66.0	**77.6**

表 2：我们的方法在高级攻击检测上优于基线方法。结果以百分比（%）表示。

虽然对注入触发器有效，但当触发器与文本上下文无缝融合时（如风格后门和 Hidden Killer 攻击），它会失败。RAP 通过在嵌入层引入另一个触发器来检测被投毒样本，假设干净样本的模型输出概率在目标类别中的下降比带有注入 RAP 触发器的被投

毒样本更大。

5.3 TrojAI 数据集上的结果

表 1 展示了我们方法和基线方法在 TrojAI 数据集上针对攻击的性能。这些模型使用 Badnets 攻击被投毒，条件触发器是在某些位置注入的字符、词或短语。PARAFUZZ 利用随机种子提示“sound like a young girl”，对几乎所有模型都实现了高精确率和高召回率。对于模型#46，我们的方法也具有与基线相当的性能。

STRIP 导致高假阴性率，因为其扰动方法无法确保触发器的正确放置或保持长触发器的完整性（例如，对于#39 号模型，STRIP 仅实现 2.0%的召回率）。RAP 由于在小验证集上计算的非代表性阈值以及注入的 RAP 触发器破坏了原始触发器的有效位置，难以准确检测大多数模型的被投毒样本，特别是对于长短语触发器。ONION 在基线中表现最好，但在复杂触发器或隐蔽触发器方面存在困难。例如，在#22 和#45 号模型（触发词为长短语）以及#19 号模型（触发词为单字符']'）上，ONION 的 F1 分数低于 80%，而我们的方法达到了约 90%。

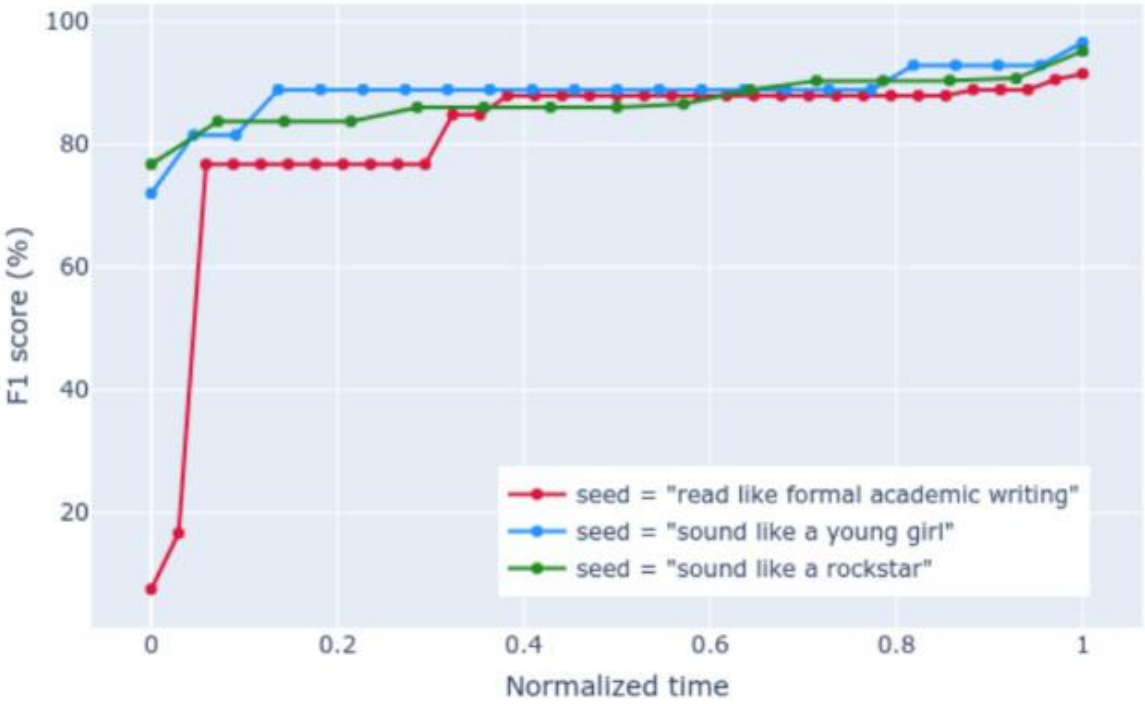

图 5：从 3 个不同的随机种子出发，在第 36 号模型上随时间推移达到的最高 F1 分数。结果表明，模糊测试的有效性与初始种子无关（即具有种子鲁棒性）。

5.4 高级攻击上的结果

表 2 展示了基线方法和我们技术在防御更高级攻击（包括 EP、风格后门和 Hidden Killer 攻击）方面的结果。对于 EP，ONION 和我们的方法实现了相当好的性能；RAP 和 STRIP 的性能再次受到验证集小尺寸的限制。

在风格后门攻击中，触发器（如圣经风格、莎士比亚风格）通过几个元素传达，其中之一是词汇。例如，莎士比亚风格倾向于使用老式的词。ONION 和 STRIP 可能会移除/替换部分基本词。尽管如此，它们未能删除风格中的其他元素，如句子结构和语调。RAP 对验证集的大小敏感，也未能有效检测被投毒样本。

Hidden Killer 是最隐蔽的攻击，因为它不涉及词汇作为触发器的症状。因此，所有 3 个基线都无法检测被 Hidden Killer 投毒的样本。我们的技术成功处理了这两种类型的攻击，并展示了跨任务的通用性。

第 6 章　种子消融研究

在本节中，我们使用模型#36 作为随机选择的主题，展示我们模糊测试技术的有效性是与种子无关的。我们随机选择 3 个由 ChatGPT 生成的种子提示，引导命令为：“列出 10 种可以应用于文本以产生不同效果的独特风格。”我们将模糊测试终止条件设置为当前最高 F1 分数超过 95%或突变体总数超过 300。

我们在包含 50 个干净样本和 50 个带有真实触发器的被投毒样本的验证集上启动模糊测试过程，并记录随时间达到的最大 F1 分数。请注意，我们对时间进行了归一化，因为种子需要不同的时间来终止模糊测试过程。尽管从不同的 F1 分数开始，所有三个种子最终都突变产生了超过 90%的 F1 分数来检测被投毒样本。结果表明我们模糊测试技术的有效性是与种子无关的。

第 7 章　结论

在本文中，我们介绍了一种用于检测 NLP 模型中被投毒样本的测试时框架，利用模型可解释性来增强后门防御。使用 ChatGPT 进行释义，我们将触发器移除转化为提示工程任务，并应用模糊测试来寻找最优释义提示。我们的实验表明，我们的方法优于当前方法，特别是针对隐蔽攻击如 Hidden Killer 攻击。

参考文献

[1] Azizi A, Tahmid I A, Waheed A, et al. T-miner: A generative approach to defend against trojan attacks on dnn-based text classification[C]//30th USENIX Security Symposium. 2021.

[2] Chen C, Dai J. Mitigating backdoor attacks in lstm-based text classification systems by backdoor keyword identification[J]. Neurocomputing, 2021, 452: 253-262.

[3] Chen X, Salem A, Chen D, et al. Badnl: Backdoor attacks against nlp models with semantic-preserving improvements[C]//ACSAC. 2021: 554-569.

[4] Cui G, Yuan L, He B, et al. A unified evaluation of textual backdoor learning: Frameworks and benchmarks[J]. Advances in Neural Information Processing Systems, 2022, 35: 5009-5023.

[5] Dai J, Chen C, Li Y. A backdoor attack against lstm-based text classification systems[J]. IEEE Access, 2019, 7.

[6] Gao Y, Kim Y, Doan B G, et al. Design and evaluation of a multi-domain trojan detection method on deep neural networks[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2349-2364.

[7] Gu T, Dolan-Gavitt B, Garg S. Badnets: Identifying vulnerabilities in the machine learning model supply chain[J]. arXiv preprint arXiv:1708.06733, 2017.

[8] Kurita K, Michel P, Neubig G. Weight poisoning attacks on pre-trained models[C]//ACL. 2020.

[9] Li L, Song D, Li X, et al. Backdoor attacks on pre-trained models by layerwise weight poisoning[C]//EMNLP. 2021: 3023-3032.

[10] Li S, Liu H, Dong T, et al. Hidden backdoors in human-centric language models[C]//CCS. 2021: 3123-3140.

[11] Liu Y, Shen G, Tao G, et al. Piccolo: Exposing complex backdoors in nlp transformer models[C]//2022 IEEE Symposium on Security and Privacy. IEEE, 2022: 2025-2042.

[12] Ni J, Li J, McAuley J. Justifying recommendations using distantly-labeled reviews and fine-grained aspects[C]//EMNLP-IJCNLP. 2019: 188-197.

[13] Pan X, Zhang M, Sheng B, et al. Hidden trigger backdoor attack on nlp models via linguistic style manipulation[C]//USENIX Security. 2022: 3611-3628.

[14] Qi F, Chen Y, Li M, et al. Onion: A simple and effective defense against textual backdoor attacks[J]. arXiv preprint arXiv:2011.10369, 2020.

[15] Qi F, Chen Y, Zhang X, et al. Mind the style of text! adversarial and backdoor attacks

based on text style transfer[C]//EMNLP. 2021: 4569-4580.

[16] Qi F, Li M, Chen Y, et al. Hidden killer: Invisible textual backdoor attacks with syntactic trigger[C]//ACL/IJCNLP. 2021.

[17] Qi F, Yao Y, Xu S, et al. Turn the combination lock: Learnable textual backdoor attacks via word substitution[C]//ACL/IJCNLP. 2021: 4873-4883.

[18] Shen G, Liu Y, Tao G, et al. Constrained optimization with dynamic bound-scaling for effective nlp backdoor defense[C]//International Conference on Machine Learning. PMLR, 2022: 19879-19892.

[19] Shen L, Ji S, Zhang X, et al. Backdoor pre-trained models can transfer to all[J]. arXiv preprint arXiv:2111.00197, 2021.

[20] Yang W, Li L, Zhang Z, et al. Be careful about poisoned word embeddings: Exploring the vulnerability of the embedding layers in nlp models[J]. arXiv preprint arXiv:2103.15543, 2021.

[21] Yang W, Lin Y, Li P, et al. Rap: Robustness-aware perturbations for defending against backdoor attacks on nlp models[J]. arXiv preprint arXiv:2110.07831, 2021.

[22] Yang W, Lin Y, Li P, et al. Rethinking stealthiness of backdoor attack against nlp models[C]//ACL. 2021: 5543-5557.

[23] Zhang X, Zhao J, LeCun Y. Character-level convolutional networks for text classification[J]. Advances in neural information processing systems, 2015, 28.